

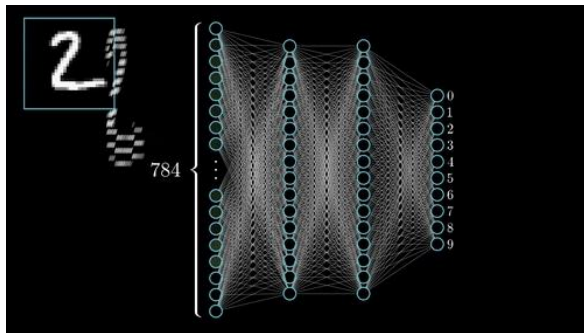
Convolutional Neural Networks

History

- In 1995, **Yann** introduced the concept of convolutional neural networks.



Problems of ANN



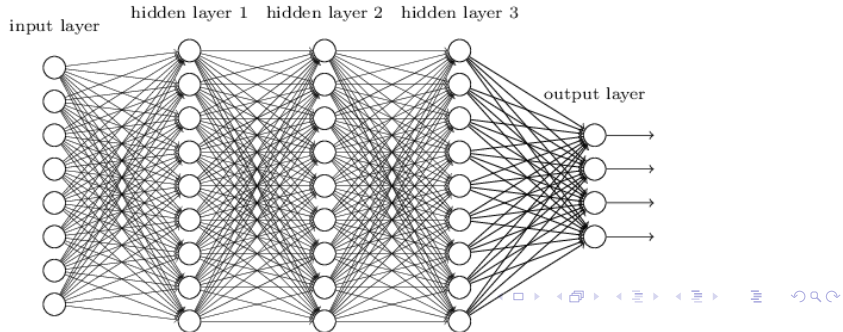
Fully connected Neural Network

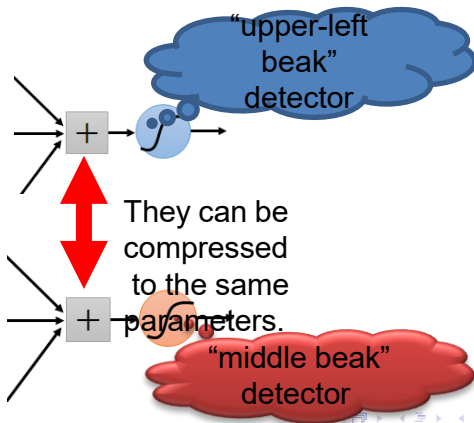
From this fully connected model, do we really need all the edges?

Can some of these be shared?

Each pixel is flattened as 1D and feed into input layer

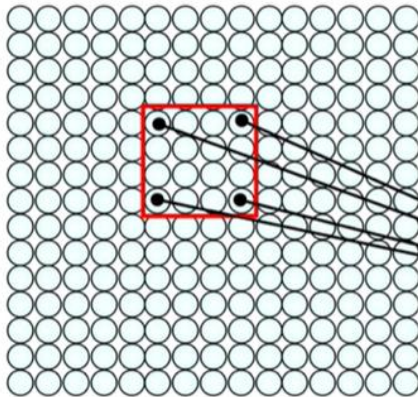
How can we use **spatial structure** in a input to inform the architecture of the network





Using Spatial Structure

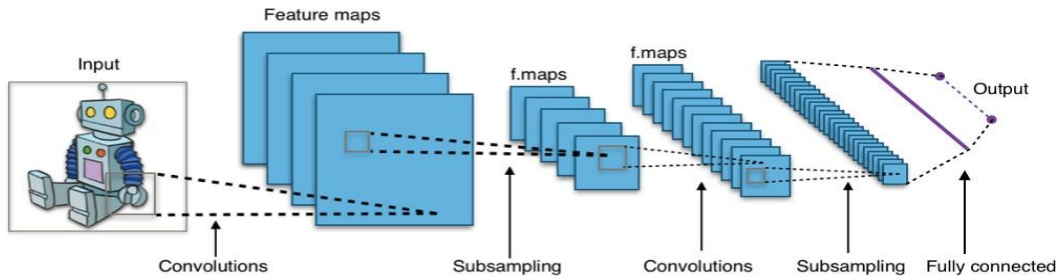
Input: 2D image.
Array of pixel values



Idea: connect patches of input
to neurons in hidden layer.

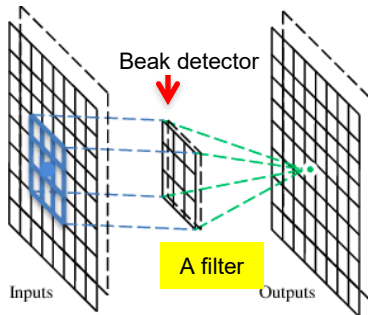
Neuron connected to region of
input. Only "sees" these values.

A typical CNN architecture



A convolutional layer

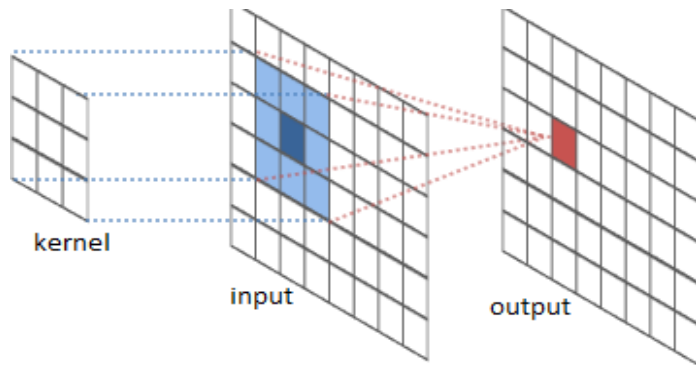
A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.



Convolution kernels

- Each convolutional layer contains **a series of filters** known as convolutional **kernels**.
- The filter is a matrix of integers that are used on a subset of the input pixel values.
- Each pixel is multiplied by the corresponding value in the kernel, then the result is summed up for a single value for simplicity representing a grid cell, like a pixel, in the output channel/feature map.
- These are linear transformations, each convolution is a type of affine function.
- When multiple convolutional kernels are applied within a convolutional layer, many channels/feature maps are created, one from each convolutional kernel.

The convolution operation



The convolution operation

- **Convolution is a mathematical operation that allows the merging of two sets of information.**
- It's a specialized kind of linear operation
- In the case of CNN, convolution is applied to the input data to filter the information and produce the **feature map**.
- This filter is also called a **kernel**



$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a} = (x * w)_t$$

Diagram illustrating the convolution operation. Arrows point from the labels "input", "convolution", and "filter" to the corresponding parts of the equation: "input" points to x_{t-a} , "convolution" points to the summation symbol $\sum$, and "filter" points to w_{-a} .

- Suppose we are tracking the position of an aeroplane using a laser sensor at discrete time intervals
- Now suppose our sensor is noisy
- To obtain a less noisy estimate we would like to average several measurements
- More recent measurements are more important so we would like to take a weighted average

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80						
---	--	--	--	--	--	------	--	--	--	--	--	--

$$S_6 = X_6 W_0 + X_5 W_{-1} + X_4 W_{-2} + X_3 W_{-3} + X_2 W_{-4} + X_1 W_{-5} + X_0 W_{-6}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80	1.96				
---	--	--	--	--	--	------	------	--	--	--	--

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_2 w_{-4} + x_1 w_{-5} + x_0 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S							1.80	1.96	2.11			
---	--	--	--	--	--	--	------	------	------	--	--	--

$$S_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_2 w_{-4} + x_1 w_{-5} + x_0 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$S_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	W_{-6}	W_{-5}	W_{-4}	W_{-3}	W_{-2}	W_{-1}	W_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S	1.80	1.96	2.11	2.16		
---	------	------	------	------	--	--

$$S_6 = X_6W_0 + X_5W_{-1} + X_4W_{-2} + X_3W_{-3} + X_2W_{-4} + X_1W_{-5} + X_0W_{-6}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6} w_{-5} w_{-4} w_{-3} w_{-2} w_{-1} w_0																								
W	<table><tr><td>0.01</td><td>0.01</td><td>0.02</td><td>0.02</td><td>0.04</td><td>0.4</td><td>0.5</td></tr></table>							0.01	0.01	0.02	0.02	0.04	0.4	0.5											
0.01	0.01	0.02	0.02	0.04	0.4	0.5																			
X	<table><tr><td>1.00</td><td>1.10</td><td>1.20</td><td>1.40</td><td>1.70</td><td>1.80</td><td>1.90</td><td>2.10</td><td>2.20</td><td>2.40</td><td>2.50</td><td>2.70</td></tr></table>													1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70														
S	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td>1.80</td><td>1.96</td><td>2.11</td><td>2.16</td><td>2.28</td><td></td></tr></table>																			1.80	1.96	2.11	2.16	2.28	
						1.80	1.96	2.11	2.16	2.28															

$$S_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_2 w_{-4} + x_1 w_{-5} + x_0 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S	1.80	1.96	2.11	2.16	2.28	2.42
---	------	------	------	------	------	------

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_2 w_{-4} + x_1 w_{-5} + x_0 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6} w_{-5} w_{-4} w_{-3} w_{-2} w_{-1} w_0						
w	0.01	0.01	0.02	0.02	0.04	0.4	0.5

x	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

s	1.80	1.96	2.11	2.16	2.28	2.42
---	------	------	------	------	------	------

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_2 w_{-4} + x_1 w_{-5} + x_0 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t
- Here the input (and the kernel) is one dimensional

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S	1.80	1.96	2.11	2.16	2.28	2.42
---	------	------	------	------	------	------

$$S_6 = X_6 W_0 + X_5 W_{-1} + X_4 W_{-2} + X_3 W_{-3} + X_2 W_{-4} + X_1 W_{-5} + X_0 W_{-6}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t
- Here the input (and the kernel) is one dimensional
- Can we use a convolutional operation on a 2D input also?

- We can think of images as 2D inputs





- We can think of images as 2D inputs
- We would now like to use a 2D filter ($m \times n$)



- We can think of images as 2D inputs
- We would now like to use a 2D filter ($m \times n$)
- First let us see what the 2D formula looks like

$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i-a, j-b} K_{a,b}$$



- We can think of images as 2D inputs
- We would now like to use a 2D filter ($m \times n$)
- First let us see what the 2D formula looks like
- This formula looks at all the preceding neighbours ($i - a, j - b$)

$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i-a, j-b} K_{a,b}$$



$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i+a, j+b} K_{a,b}$$

- We can think of images as 2D inputs
- We would now like to use a 2D filter ($m \times n$)
- First let us see what the 2D formula looks like
- This formula looks at all the preceding neighbours ($i - a, j - b$)
- In practice, we use the following formula which looks at the succeeding neighbours

- Let us apply this idea to a toy example and see the results

Input

a	b	c	d
e	f	g	h
i	j	k	/

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$		

Input

a	b	c	d
e	f	g	h
i	j	k	l

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	

Input

a	b	c	d
e	f	g	h
i	j	k	l

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	$cw+dx+gy+hz$

Input

a	b	c	d
e	f	g	h
i	j	k	l

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	$cw+dx+gy+hz$
$ew+fx+iy+jz$		

Input

a	b	c	d
e	f	g	h
i	j	k	l

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	$cw+dx+gy+hz$
$ew+fx+iy+jz$	$fw+gx+jy+kz$	

Input

a	b	c	d
e	f	g	h
i	j	k	/

Kernel

w	x
y	z

- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	$cw+dx+gy+hz$
$ew+fx+iy+jz$	$fw+gx+jy+kz$	$gw+hx+ky+lz$

Let us see some examples of 2D convolutions applied to images



$$\begin{array}{ccc} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{array} *$$

=



$$\begin{array}{ccc} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{array} =$$



blurs the image



$$\begin{matrix} * & \begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix} & = \end{matrix}$$



$$\begin{matrix} * & \begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix} & = \end{matrix}$$



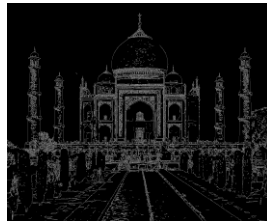
sharpens the image



$$\begin{matrix} * & \begin{matrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{matrix} & = \end{matrix}$$



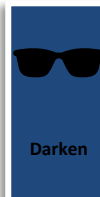
$$* \begin{matrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{matrix} =$$



detects the edges

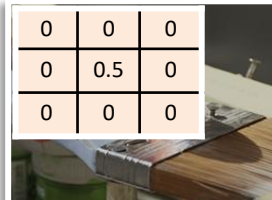
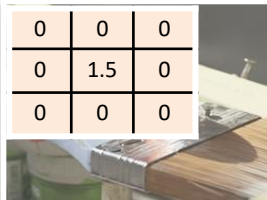
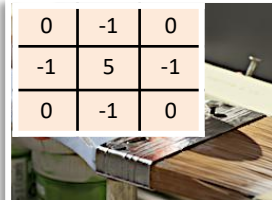
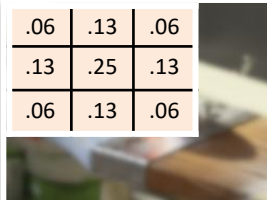


Original Image



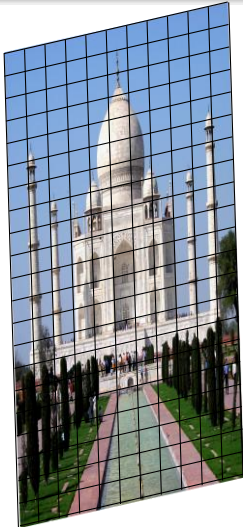


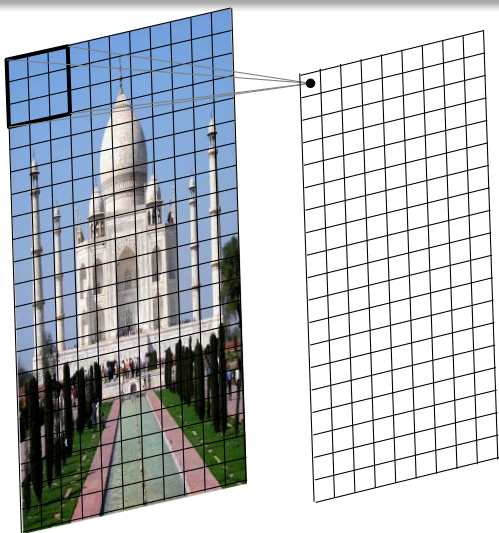
Original Image



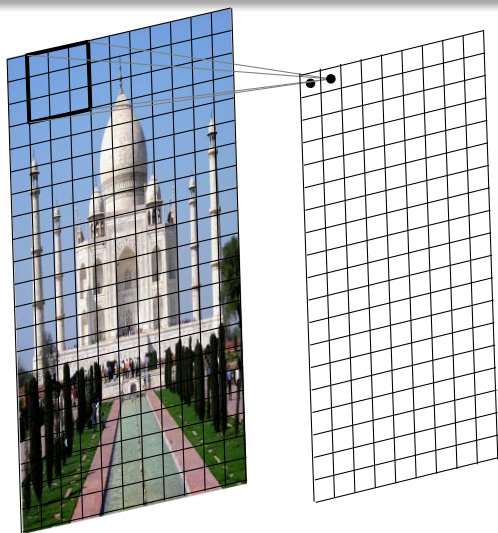
We will now see a working example of 2D convolution.

- We just slide the kernel over the input image

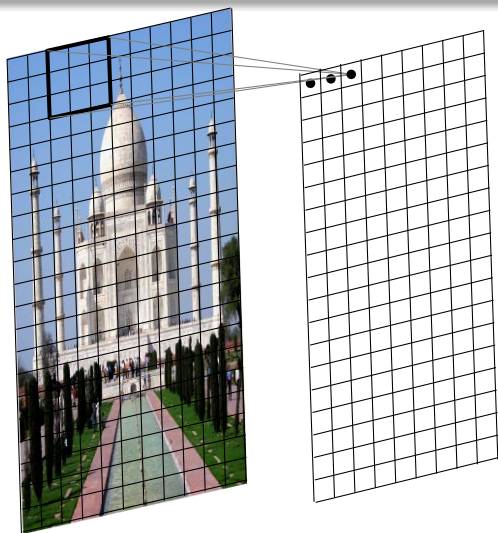




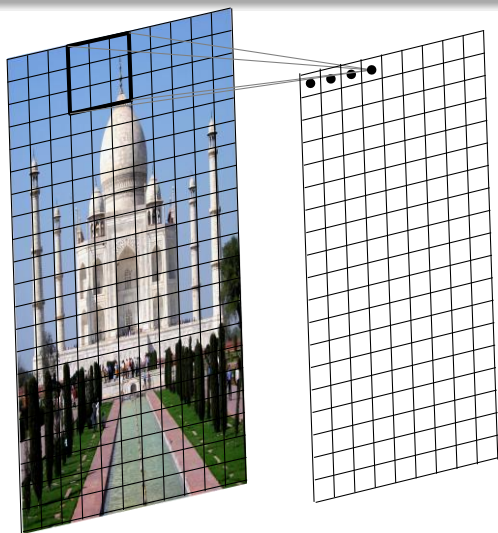
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



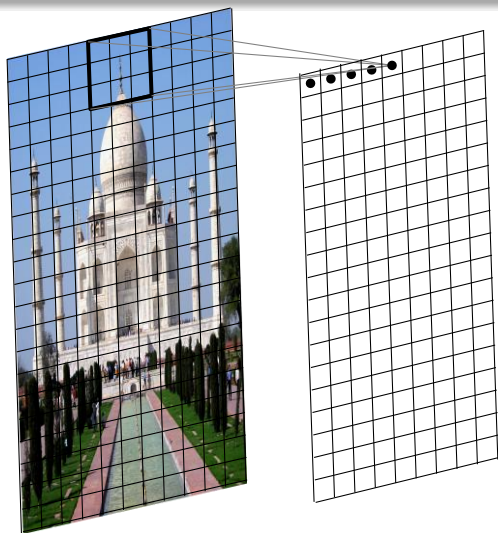
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



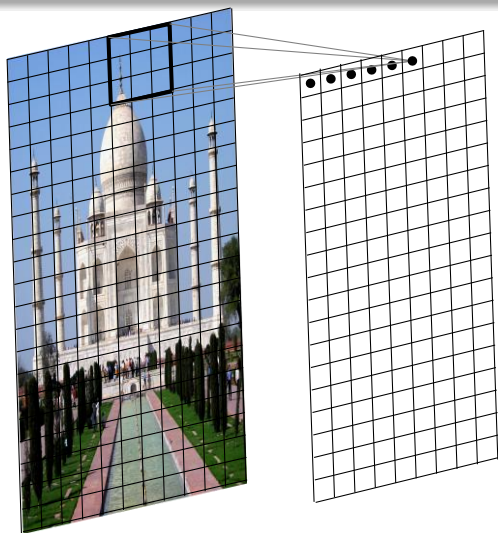
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



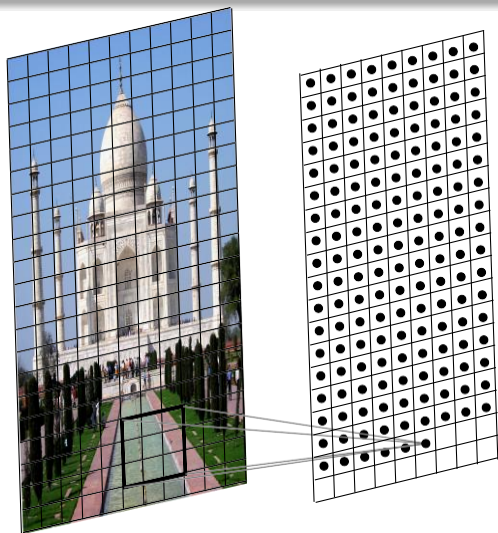
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



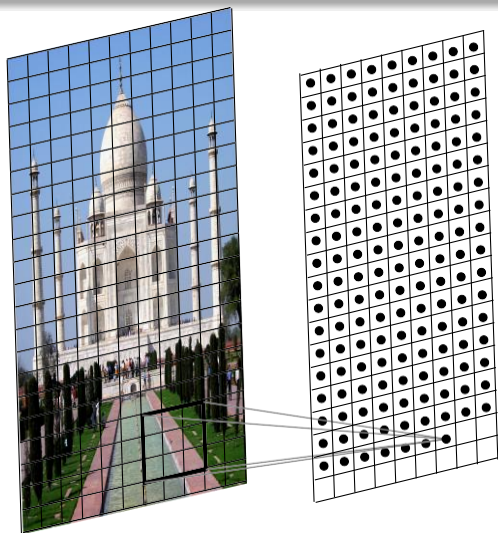
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



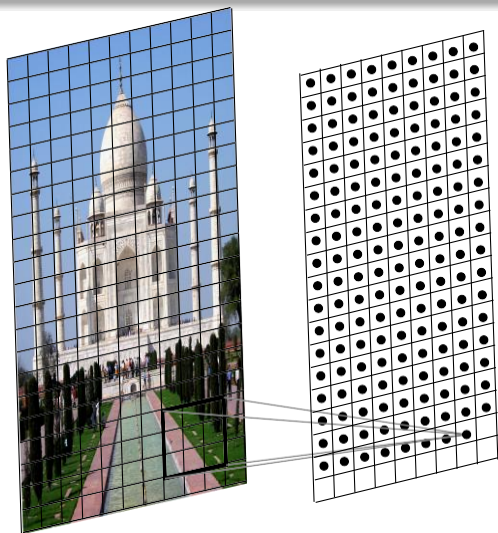
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



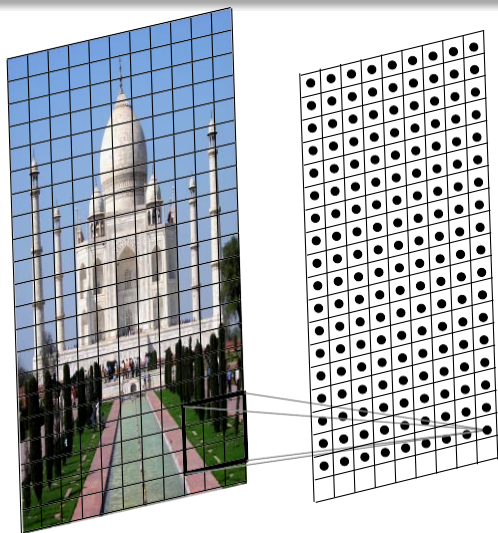
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



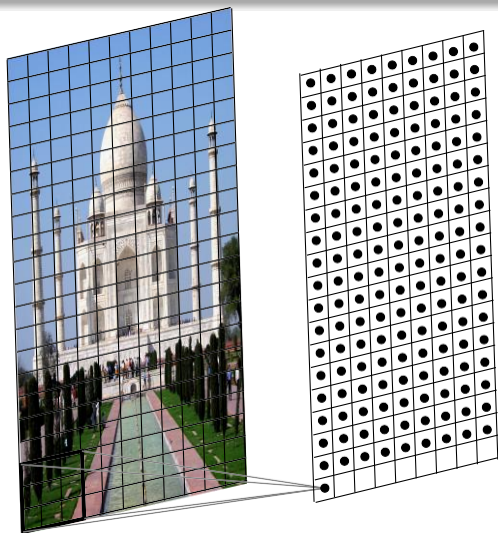
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



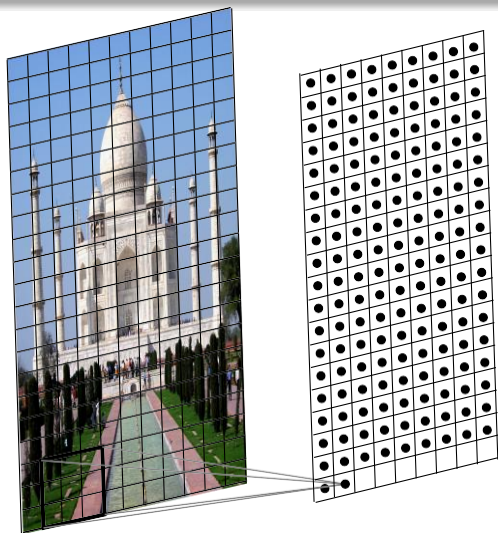
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



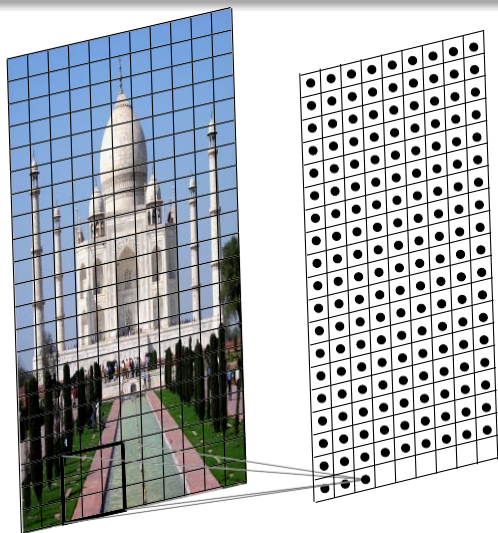
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



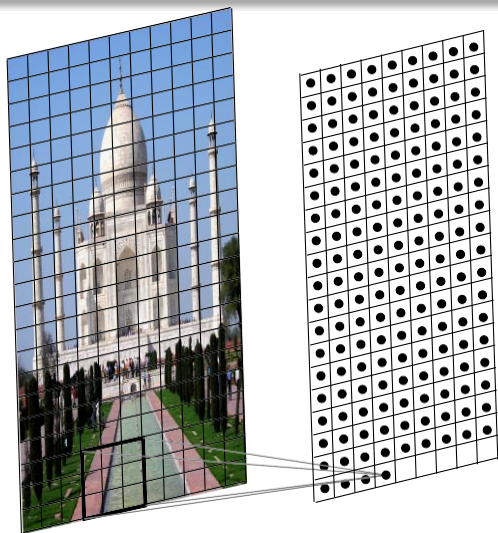
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



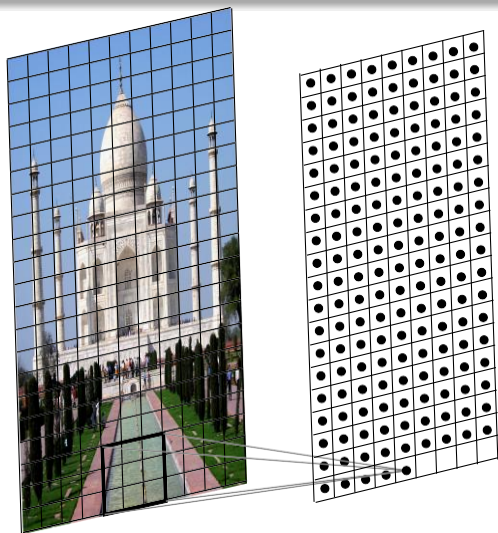
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



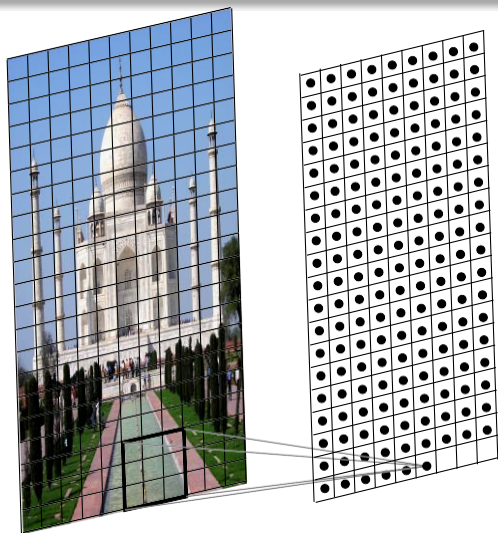
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



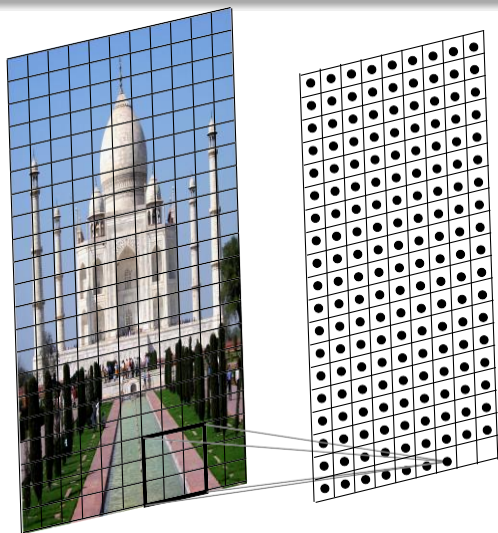
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



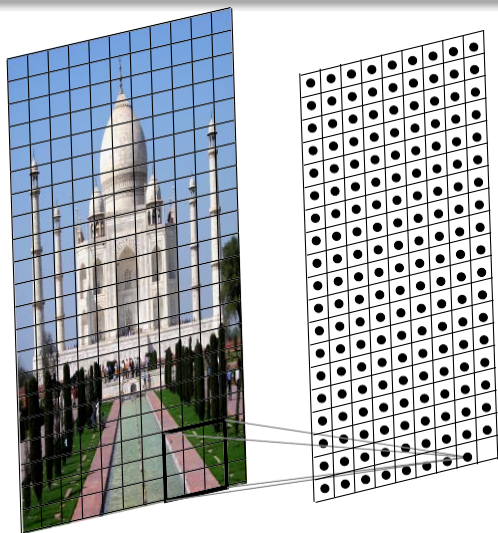
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



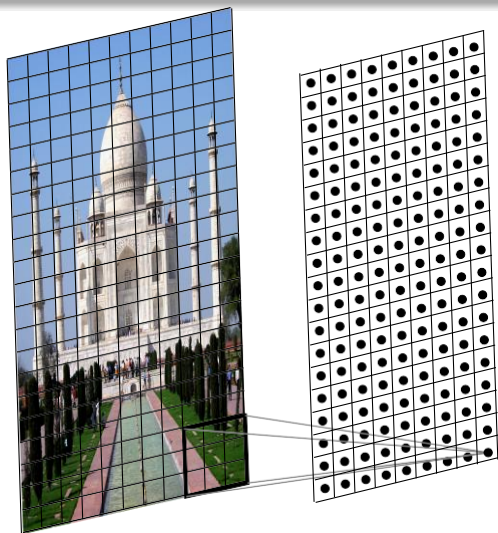
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



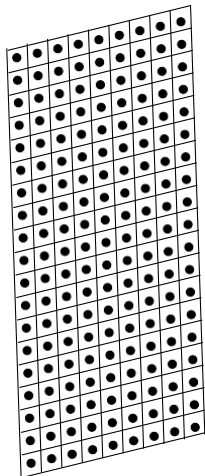
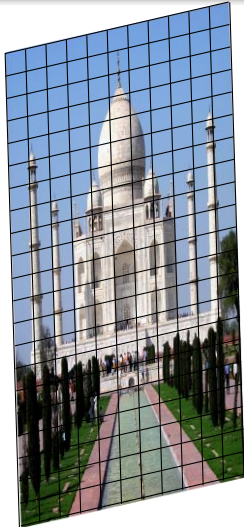
- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output
- The resulting output is called a feature map.



- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output
- The resulting output is called a feature map.
- We can use multiple filters to get multiple feature maps.

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

A	B	C	B	A	B	C
---	---	---	---	---	---	---

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

A	B	C	B	A	B	C
---	---	---	---	---	---	---

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

A	B	C	B	A	B	C
---	---	---	---	---	---	---

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

A	B	C	B	A	B	C
---	---	---	---	---	---	---

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input

A	B	C	B	A	B	C
---	---	---	---	---	---	---

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional input

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional input

a	b	c	d
e	f	g	h
i	j	k	l

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional input

a	b	c	d
e	f	g	h
i	j	k	l

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional output

a	b	c	d
e	f	g	h
i	j	k	l

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional output

a	b	c	d
e	f	g	h
i	j	k	l

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional output

a	b	c	d
e	f	g	h
i	j	k	l

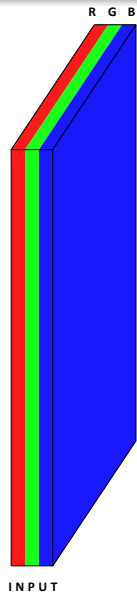
Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional input

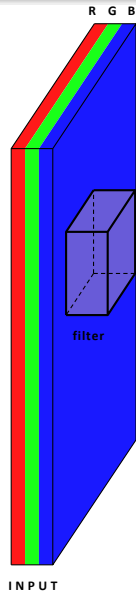
a	b	c	d
e	f	g	h
i	j	k	l

Question

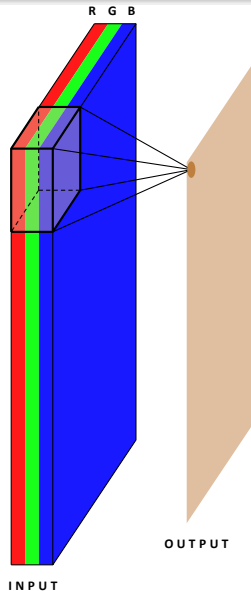
- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional output
- What would happen in the 3D case?



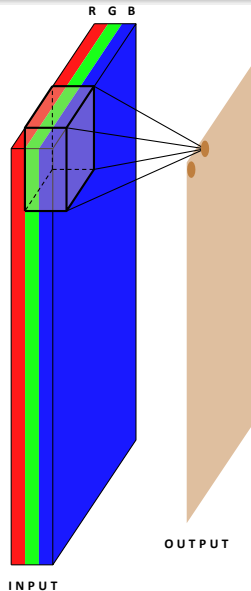
- What would a 3D filter look like?



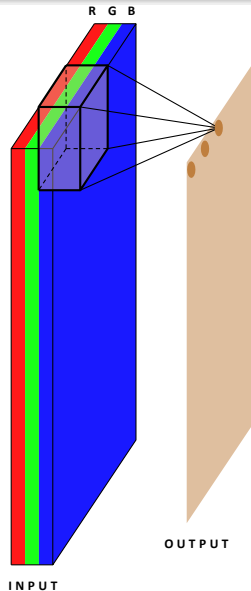
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume



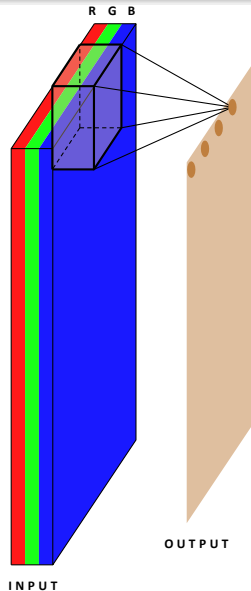
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



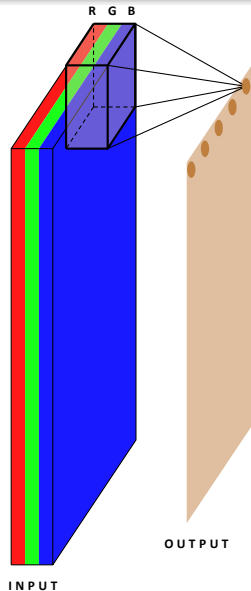
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



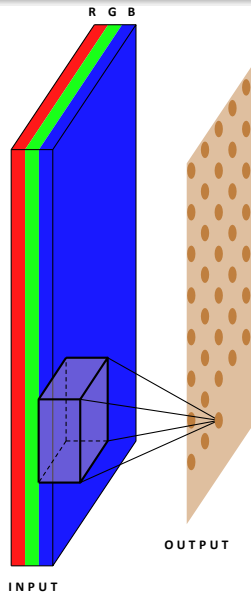
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



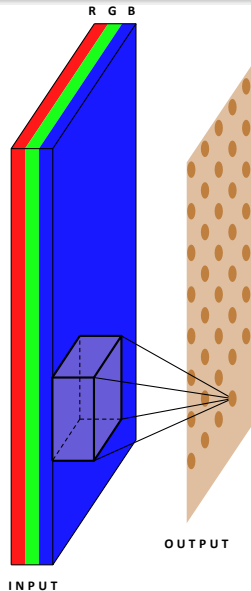
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



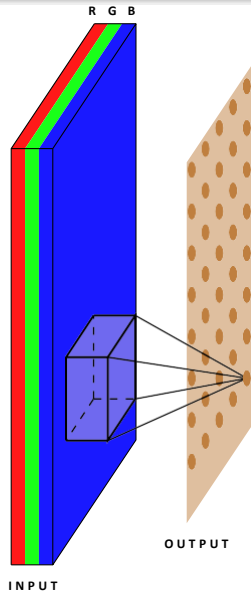
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



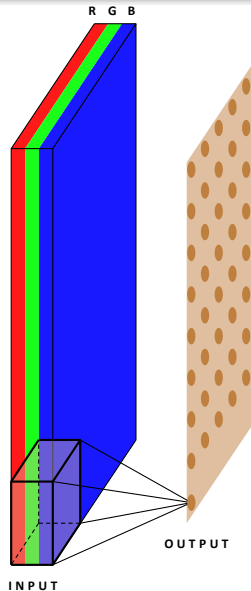
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



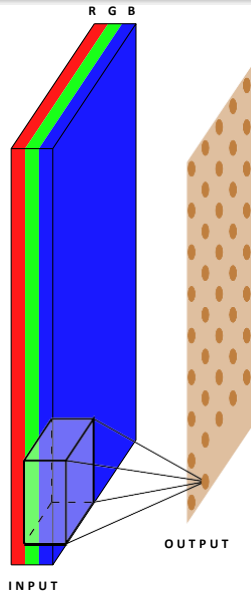
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



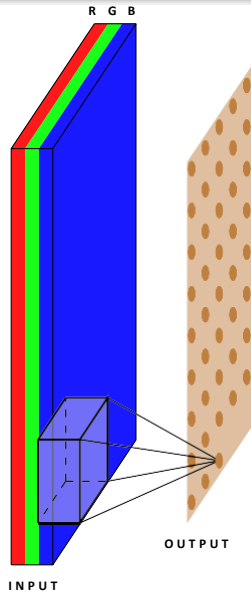
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



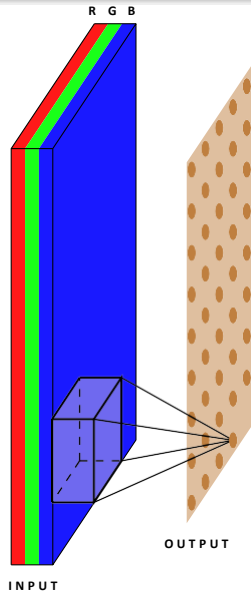
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



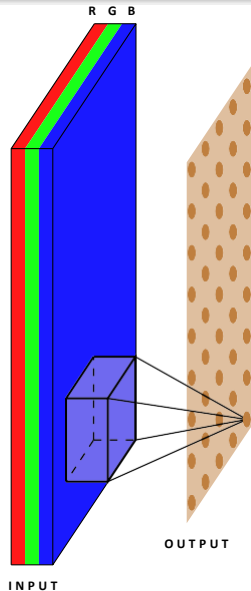
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



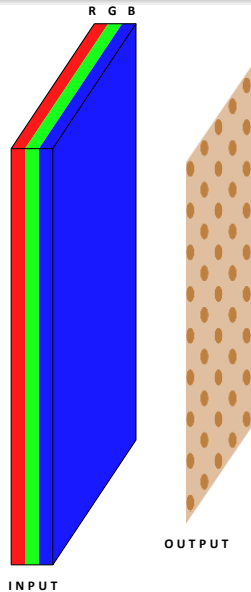
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



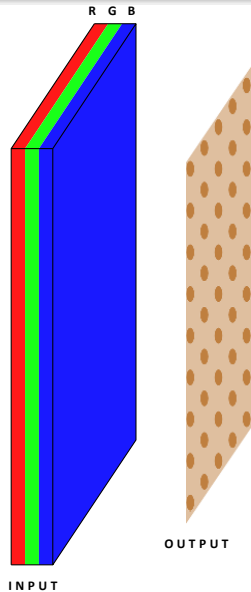
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



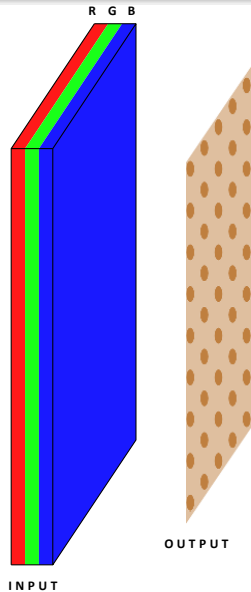
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image



- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)



- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will **be 2D** (only width and height, no depth)



- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will be 2D (only width and height, no depth)
- Once again we can apply multiple filters to get multiple feature maps